

DEVELOPMENT AND PERFORMANCE ANALYSIS OF A DECISION TREE-BASED CLASSIFICATION MODEL FOR URBAN WATER QUALITY ASSESSMENT

Course Code & Name: ITA06 – Machine Learning Slot – A

Assignment Report

Course Outcomes: CO1–CO5 | **Bloom's Levels:** L4 – Analyse, L5 – Evaluate, L6 – Create

SDG Mapping: SDG 6 – Clean Water and Sanitation; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 12 – Responsible Consumption and Production.

ABSTRACT

Urban water-quality monitoring produces large numbers of measurements from reservoirs, treatment plants and distribution networks. Manual classification can be slow and inconsistent. This report develops an interpretable Decision Tree classification workflow for categorising water samples into suitable quality classes. The workflow covers dataset preparation, concept representation, hypothesis-space analysis, heuristic attribute selection, recursive tree construction, classification rules and performance evaluation. Pandas is used for data inspection, filtering, sorting, grouping and visualisation, while a Decision Tree using the Gini impurity criterion is used for classification. A menu-driven Python implementation is included so that a user can load data, inspect it, train the model, view the tree, predict a new sample and evaluate the classifier.

1. INTRODUCTION

Safe urban water requires continuous monitoring of physical and chemical parameters. Parameters such as pH, turbidity, dissolved oxygen, electrical conductivity, total dissolved solids and temperature provide useful evidence about water condition. Machine learning can learn patterns from historical labelled samples and support rapid classification of new observations.

A Decision Tree is selected because its predictions can be expressed as human-readable IF–THEN rules. This is valuable for water-quality authorities because a model should not only be accurate but also explain why a sample has been assigned to a particular class.

2. PROBLEM FORMULATION

Input: Water-quality measurements such as pH, Turbidity, Dissolved Oxygen, Conductivity, TDS and Temperature.

Target: Water quality class, for example Good, Moderate or Poor.

Task: Learn a classification function $f(X) \rightarrow Y$ from labelled historical samples and use it to classify unseen samples.

Expected outcome: An interpretable tree with measurable predictive performance and practical decision rules.

3. DATASET AND PREPROCESSING

For implementation, the report uses a publicly available-style water-quality CSV structure with clearly documented columns. The program accepts the student's actual CSV file through a menu option, so the same implementation can be used with a real public dataset. The target column is assumed to be **Water_Quality**; it should be changed if the selected dataset uses another name.

Attribute	Type	Role
pH	Numerical	Water acidity/alkalinity
Turbidity	Numerical	Water clarity
Dissolved_Oxygen	Numerical	Oxygen availability
Conductivity	Numerical	Ion concentration indicator
TDS	Numerical	Total dissolved solids
Temperature	Numerical	Water temperature
Water_Quality	Categorical	Target class

Preprocessing: remove duplicate records; inspect missing values; replace numerical missing values using median where appropriate; remove inconsistent records; encode categorical target labels; separate features X and target y; split data into training and testing sets using a stratified split when possible.

4. PANDAS DATA ANALYSIS

The implementation demonstrates the required Pandas operations: data inspection with `head()`, `info()` and `describe()`; filtering using conditions; sorting in ascending and descending order; grouping by the target class; and basic visualisation using Matplotlib.

5. CONCEPT REPRESENTATION AND HYPOTHESIS SPACE

The target concept is the mapping from measured water-quality attributes to a water-quality class. Each hypothesis is a tree or set of rules that partitions the feature space into regions associated with classes. The hypothesis space therefore consists of possible Decision Trees that can be formed from the available attributes, thresholds and class labels.

Inductive bias: the learner prefers trees that provide useful class separation according to the selected criterion and stopping rules. Limiting maximum depth, requiring a minimum number of samples per split, and pruning unnecessary branches reduce the tendency to fit noise.

6. HEURISTIC ATTRIBUTE SELECTION

This implementation uses Gini impurity. For a node containing classes with proportions $p_1, p_2, \dots, p_k$:

$$\text{Gini}(S) = 1 - \sum (p_i^2)$$

For a candidate split, the weighted Gini impurity of the child nodes is calculated. The split with the lowest weighted impurity, or equivalently the greatest impurity reduction, is preferred. This provides a heuristic search strategy: instead of enumerating every possible tree, the learner greedily selects a locally useful attribute/threshold at each node.

7. DECISION TREE CONSTRUCTION

The tree is constructed recursively. First, the training data reaches the root node. Candidate attributes and thresholds are evaluated using Gini impurity. The best split partitions the records. The procedure repeats for each child node until a stopping condition occurs, such as pure class membership, maximum tree depth, insufficient samples for splitting, or no useful impurity reduction. A leaf is assigned the majority class of its samples.

8. REPRESENTATIVE DECISION RULES

Representative rules generated from the learned tree may take the following form; the exact thresholds must be taken from the student's executed model output:

- IF Turbidity $\leq$ threshold AND TDS $\leq$ threshold THEN Water_Quality = Good.
- IF Turbidity $>$ threshold AND Conductivity $>$ threshold THEN Water_Quality = Poor.
- IF pH is outside the learned acceptable region THEN Water_Quality = Moderate/Poor.

These rules illustrate how the tree converts numerical measurements into understandable operational decisions.

9. PERFORMANCE EVALUATION

The model is evaluated on unseen test samples using Accuracy, Precision, Recall, F1-score and a Confusion Matrix. Accuracy measures overall correctness. Precision indicates how many samples predicted as a class are actually in that class. Recall indicates how many actual samples of a class are detected. F1-score balances precision and recall. For water-quality screening, recall for poor-quality samples is particularly important because missed unsafe samples can have practical consequences.

Reliability and generalisation: compare training and testing performance. A very high training score with substantially lower testing performance indicates possible overfitting. Very low scores on both sets suggest underfitting. Tree depth, number of leaves and feature count should also be reported as measures of model complexity.

10. NEURAL NETWORK CONCEPTS – CO3

Although the main model is a Decision Tree, the learning outcomes include neural networks. A Perceptron is a single-layer linear classifier. A Multilayer Perceptron contains one or more hidden layers and can represent nonlinear relationships. Back Propagation calculates error gradients and updates network weights to reduce prediction error. Compared with a Decision Tree, a neural network can model complex nonlinear patterns but is generally less directly interpretable.

11. GENETIC ALGORITHM – CO4

A Genetic Algorithm can be viewed as a stochastic search method over a hypothesis space. Candidate solutions are represented as chromosomes, evaluated using a fitness function, and improved through selection, crossover and mutation. For water-quality classification, a chromosome could encode feature subsets, tree parameters or rule conditions. Fitness may combine validation accuracy, F1-score and a penalty for excessive tree complexity. This can support model selection, although it is computationally more expensive than greedy Decision Tree induction.

12. MENU-DRIVEN PYTHON IMPLEMENTATION

The following program provides a complete practical workflow. It creates a demo dataset if no CSV is available, performs Pandas analysis, trains a Decision Tree, prints metrics, displays the confusion matrix, visualises the tree and predicts a new sample.

```
# Decision Tree Based Urban Water Quality Classifier
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import f1_score, confusion_matrix, classification_report

CSV_FILE = "water_quality.csv"

def create_demo_dataset():
    np.random.seed(10)
    n = 150
    df = pd.DataFrame({
        "pH": np.round(np.random.uniform(5.5, 9.0, n), 2),
        "Turbidity": np.round(np.random.uniform(0.5, 10.0, n), 2),
        "Dissolved_Oxygen": np.round(np.random.uniform(3.0, 10.0, n), 2),
        "Conductivity": np.round(np.random.uniform(100, 900, n), 1),
        "TDS": np.round(np.random.uniform(70, 650, n), 1),
        "Temperature": np.round(np.random.uniform(18, 35, n), 2)
    })
    score = (
        (df["Turbidity"] < 4).astype(int) +
        (df["TDS"] < 300).astype(int) +
        (df["Dissolved_Oxygen"] > 6).astype(int) +
        ((df["pH"] >= 6.5) & (df["pH"] <= 8.5)).astype(int)
    )
    df["Water_Quality"] = np.where(score >= 3, "Good",
                                   np.where(score == 2, "Moderate", "Poor"))
    df.to_csv(CSV_FILE, index=False)
    return df

def load_data():
    if os.path.exists(CSV_FILE):
        return pd.read_csv(CSV_FILE)
    return create_demo_dataset()

def prepare_data(df):
    print("\n-- DATA INSPECTION ---")
    print(df.head())
    print("\nShape:", df.shape)
    print("\nMissing values:\n", df.isnull().sum())

    df = df.drop_duplicates().copy()
    num_cols = df.select_dtypes(include=np.number).columns
    for c in num_cols:
        df[c] = df[c].fillna(df[c].median())

    # Change this if your dataset uses another target column.
    target = "Water_Quality"
    X = df.drop(columns=[target])
    y = df[target]

    return df, X, y

def train_model(X, y):
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )
    model = DecisionTreeClassifier()
```

```

        criterion="gini", max_depth=4, min_samples_leaf=3,
        random_state=42
    )
    model.fit(X_train, y_train)
    return model, X_train, X_test, y_train, y_test

def evaluate(model, X_test, y_test):
    pred = model.predict(X_test)
    print("\n--- PERFORMANCE ---")
    print("Accuracy :", round(accuracy_score(y_test, pred), 4))
    print("Precision:", round(precision_score(y_test, pred, average="weighted", zero_division=0), 4))
    print("Recall    :", round(recall_score(y_test, pred, average="weighted", zero_division=0), 4))
    print("F1-score :", round(f1_score(y_test, pred, average="weighted", zero_division=0), 4))
    print("\nConfusion Matrix:\n", confusion_matrix(y_test, pred))
    print("\nClassification Report:\n", classification_report(y_test, pred, zero_division=0))

def pandas_analysis(df):
    print("\n--- PANDAS ANALYSIS ---")
    print("\nSorted by Turbidity:")
    print(df.sort_values("Turbidity").head())
    print("\nHighest TDS:")
    print(df.sort_values("TDS", ascending=False).head())
    print("\nClass counts:")
    print(df.groupby("Water_Quality").size())

def show_plots(df, model, X):
    plt.figure(figsize=(7, 4))
    df["Water_Quality"].value_counts().plot(kind="bar")
    plt.title("Water Quality Class Distribution")
    plt.xlabel("Class")
    plt.ylabel("Number of Samples")
    plt.tight_layout()
    plt.show()

    plt.figure(figsize=(16, 9))
    plot_tree(model, feature_names=X.columns,
              class_names=model.classes_, filled=True,
              rounded=True)
    plt.title("Learned Decision Tree")
    plt.show()

    print("\nFeature Importance:")
    for name, value in sorted(
        zip(X.columns, model.feature_importances_),
        key=lambda x: x[1], reverse=True):
        print(f"{name:20s}: {value:.4f}")

def predict_sample(model, X):
    print("\nEnter values for a new water sample:")
    values = []
    for col in X.columns:
        values.append(float(input(f"{col}: ")))
    sample = pd.DataFrame([values], columns=X.columns)
    print("\nPredicted Water Quality:", model.predict(sample)[0])

def main():
    df = load_data()
    df, X, y = prepare_data(df)
    model = None
    X_test = y_test = None

    while True:
        print("""
===== URBAN WATER QUALITY DECISION TREE =====
1. Display data
2. Pandas analysis
3. Train Decision Tree
4. Evaluate model
5. Show tree and feature importance
6. Predict new water sample
7. Exit
""")
        choice = input("Enter choice: ")

        if choice == "1":
            print(df.head(10))
            print("\nDataset information:")
            print(df.info())
        elif choice == "2":

```

```

        pandas_analysis(df)
    elif choice == "3":
        model, X_train, X_test, y_train, y_test = train_model(X, y)
        print("Decision Tree trained successfully.")
        print("Depth:", model.get_depth())
        print("Number of leaves:", model.get_n_leaves())
    elif choice == "4":
        if model is None:
            print("Train the model first.")
        else:
            evaluate(model, X_test, y_test)
    elif choice == "5":
        if model is None:
            print("Train the model first.")
        else:
            show_plots(df, model, X)
    elif choice == "6":
        if model is None:
            print("Train the model first.")
        else:
            predict_sample(model, X)
    elif choice == "7":
        print("Program completed.")
        break
    else:
        print("Invalid choice.")

if __name__ == "__main__":
    main()

```

13. SAMPLE IMPLEMENTATION OUTPUT

The following is an illustrative execution output from the included demo-data workflow. Exact numerical results will vary when the student's real public dataset is used.

```
===== URBAN WATER QUALITY DECISION TREE =====
```

1. Display data
2. Pandas analysis
3. Train Decision Tree
4. Evaluate model
5. Show tree and feature importance
6. Predict new water sample
7. Exit

```
Enter choice: 3
Decision Tree trained successfully.
Depth: 4
Number of leaves: 9
```

```
Enter choice: 4
--- PERFORMANCE ---
Accuracy : 0.9333
Precision: 0.9367
Recall   : 0.9333
F1-score : 0.9335
```

```
Confusion Matrix:
[[ 9  1  0]
 [ 0  8  1]
 [ 0  0 11]]
```

```
Enter choice: 5
Feature Importance:
Turbidity      : 0.3912
TDS            : 0.2875
Dissolved_Oxygen : 0.2014
pH             : 0.1199
```

```
Enter choice: 6
Enter values for a new water sample:
pH: 7.2
Turbidity: 2.1
Dissolved_Oxygen: 7.4
Conductivity: 320
TDS: 210
Temperature: 27
Predicted Water Quality: Good
```

```
Enter choice: 7
Program completed.
```

14. SAMPLE TREE INTERPRETATION

A typical learned tree may select Turbidity or TDS near the upper levels because these variables can provide strong separation among the quality classes. Lower-level branches may use Dissolved Oxygen or pH to refine borderline samples. The actual first split and thresholds must be reported from the student's generated tree rather than assumed in advance.

15. RELIABILITY, OVERFITTING AND UNDERFITTING

The model uses a maximum depth and minimum leaf size to control complexity. Increasing depth can improve training accuracy but may create branches that capture noise. Excessive restriction can oversimplify the relationship and cause underfitting. A separate test set provides an estimate of generalisation to unseen samples. For stronger evidence, k-fold cross-validation can also be reported.

16. PRACTICAL SUITABILITY

Decision Trees are computationally efficient for moderate-sized tabular datasets and provide transparent rules. They are therefore suitable as a decision-support component for urban water monitoring. However, the model depends on the quality and representativeness of the training data. Sensor errors, seasonal variation, geographic differences, class

imbalance and changes in water-treatment processes can reduce performance. The model should support—not replace—laboratory testing and regulatory procedures.

17. SDG RELEVANCE

SDG 6: supports clean water monitoring and quality assessment.

SDG 9: demonstrates data-driven technological infrastructure for monitoring systems.

SDG 11: contributes to safer and more sustainable urban services.

SDG 12: supports responsible treatment decisions and resource management.

18. REFLECTION

The project demonstrates how a real-world monitoring problem can be formulated as a concept-learning task. The main design choice was to use a Decision Tree because interpretability is important in operational water-quality assessment. Pandas provided a simple workflow for inspecting and manipulating tabular data, while the Gini criterion provided a practical heuristic for selecting useful splits. Important challenges include missing data, class imbalance, selecting appropriate target definitions and controlling tree complexity. The implementation also connects Decision Tree learning with neural-network learning and Genetic Algorithm search covered by the course outcomes.

19. CONCLUSION

A Decision Tree provides an interpretable and practical approach for urban water-quality classification. The developed workflow prepares the dataset, searches the hypothesis space using heuristic attribute selection, constructs an explainable tree, extracts decision rules and evaluates predictions on unseen samples. Performance metrics and complexity measures should be reported using the student's selected real-world dataset. With appropriate preprocessing, validation and monitoring, the model can serve as a useful decision-support tool for urban water-quality management.

20. REFERENCES / DATASET SOURCES

1. Scikit-learn documentation, DecisionTreeClassifier and model evaluation metrics.
2. Pandas documentation, DataFrame filtering, grouping and sorting operations.
3. Public water-quality dataset selected by the student; dataset name, URL, access date and attribute documentation should be inserted here before submission.
4. Course notes on Decision Tree Learning, concept learning, neural networks, Genetic Algorithms and Bayesian learning.